

A Comparative Performance Analysis of 4-Stage and 6-Stage RISC-V Cores in a Multicore FPGA System

Project report submitted in partial fulfillment
of the requirements for the degree of

Bachelor of Technology
in
Electronics and Communication Engineering

by

Molik Rajvanshi — Roll No. 23UEC575
Ujjawal Khatri — Roll No. 23UEC635

Under Guidance of
Dr. Abhishek Sharma



Department of Electronics and Communication Engineering
The LNM Institute of Information Technology, Jaipur

2024-25

© 2025 Molik Rajvanshi and Ujjawal Khatri

The LNM Institute of Information Technology (LNMIIT), Jaipur

All rights reserved.

The LNM Institute of Information Technology
Jaipur, India

CERTIFICATE

This is to certify that the project entitled “A Comparative Performance Analysis of 4-Stage and 6-Stage RISC-V Cores in a Multicore FPGA System”, submitted by Molik Rajvanshi (Roll no 23UEC575) and Ujjawal Khatri (Roll no 23UEC635) in partial fulfillment of the requirement of degree in Bachelor of Technology (B. Tech), is a bonafide record of work carried out by them at the Department of Electronics and Communication Engineering, The LNM Institute of Information Technology, Jaipur, (Rajasthan) India, during the academic session 2024-2025 under my supervision and guidance and the same has not been submitted elsewhere for award of any other degree. In my/our opinion, this report is of standard required for the award of the degree of Bachelor of Technology (B. Tech).

Date

Adviser: Dr. Abhishek Sharma

Acknowledgments

We would like to express our sincere gratitude to our faculty mentor **Dr. Abhishek Sharma**, Assistant Professor, Department of Electronics and Communication Engineering, LNMIIT Jaipur, for his invaluable guidance, support, and encouragement throughout this project.

We are also thankful to the Department of Electronics and Communication Engineering, LNMIIT, for providing us the necessary facilities and resources to carry out this work.

Molik Rajvanshi (23UEC575)

Ujjawal Khatri (23UEC635)

Abstract

This report presents multicore scaling and benchmarking of two families of lightweight RV32I soft-core processors running on a PYNQ-Z2 (Zynq-7020) FPGA. The designs include a 4-stage pipelined core scaled to 2, 4, and 8 cores, and a 6-stage multicycle multicore architecture. The cores execute a uniform bare-metal workload without libc library support and utilise a mailbox-based synchronisation scheme along with shared BRAM. The data memory is partitioned per core to maintain architectural parity across all cores. Additionally, the performance is evaluated using MiBench integer benchmarks such as Basicmath, Bitcount, Qsort, Stringsearch, FFT, and IFFT. Experimental results demonstrate that the 4-stage multicore architecture achieves much higher performance and efficiency across all core counts than multicycle cores. The multicore speedup is up to 5.98 times for the 8-core configuration. Furthermore, the 4-stage architecture demonstrates superior resource utilization, consuming only 28,591 LUTs and 1.63 W of total power in the 8-core configuration, compared to 43,731 LUTs and 1.69 W for the 6-stage design. As a practical application, a lightweight Convolutional Neural Network (CNN) for fruit image classification is deployed on the 4-core system using int8 quantized inference, correctly classifying all five fruit categories and validating the architecture for embedded AI workloads.

Keywords: RISC-V, multicore processors, FPGA, parallel performance, embedded systems, CNN inference, int8 quantization

Contents

List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Research Gap and Novel Contributions	2
1.2 Scope of This Work	2
2 Literature Review	4
3 Proposed Work	6
3.1 System Architecture Overview	6
3.2 Software Stack and Build Flow (Bare-Metal)	7
3.3 Memory and Interconnect Organization	7
3.4 Firmware Execution, Host Orchestration and Measurement	8
3.5 Timing Closure	8
3.6 Benchmarks, Inputs and Benchmarking Metrics	10
4 Simulation and Results	11
4.1 Experiment Setup	11
4.2 Results and Discussion	11
4.3 FPGA Resource Utilization and Power	13
4.4 Application — CNN-Based Fruit Classification on the Multicore System	14
4.4.1 Overview and Motivation	14
4.4.2 CNN Model Architecture	14
4.4.3 Int8 Quantization and Calibration Pipeline	15
4.4.4 Deployment Pipeline	16
4.4.5 Host–Worker Execution for CNN Inference	16
4.4.6 FPGA Classification Results	18
4.4.7 Key Technical Challenges and Solutions	18
5 Discussion	20
5.1 Discussion and Comparison with Related Work	20
5.2 CNN Inference as a Workload Characterization	21
5.3 Towards an AI Accelerator: Architectural Extensions	22
5.3.1 Custom Multiply-Accumulate (MAC) Instruction	22
5.3.2 SIMD Int8 Packed Operations	23
5.3.3 Systolic Array Co-Processor	23

5.3.4	Hardware Activation Functions	24
5.3.5	DMA-Based Data Streaming	24
5.3.6	Binary Neural Networks for Multiply-Free Inference	25
6	Conclusions and Future Work	26
6.1	Summary of Contributions	26
6.2	Limitations	27
6.3	Future Work	27
	Bibliography	28

List of Figures

3.1	Architecture of the proposed RV32I multicore FPGA system. The Zynq PS acts as host controller and communicates with multiple RV32I soft cores through an AXI interconnect. Each core accesses a private BRAM window containing mailbox synchronization structures and local data memory.	6
3.2	2-Core Block Diagram as seen in the Vivado block design.	8
4.1	Total MIPS comparison using achieved clocks (higher is better).	13
4.2	Scaling efficiency relative to the 2-core baseline of each family.	13
4.3	Deployment pipeline from PyTorch training to on-FPGA int8 CNN inference.	16
4.4	Banana 1 classification result on FPGA hardware.	19
4.5	Pomegranate 1 classification result on FPGA hardware.	19

List of Tables

2.1	Lightweight RV32 soft-cores relevant to this paper (non-Linux class). . .	5
3.1	Post-implementation timing results for both processor families across different core counts.	8
4.1	Platform, tools, and run invariants.	12
4.2	Complete performance comparison across all kernels and configurations at achieved frequencies. Speedup and efficiency are relative to the 2-core baseline of the same family.	12
4.3	Post-implementation FPGA utilization and total power for all multicore configurations on the PYNQ-Z2 (Zynq-7020).	14
4.4	FruitCNNSmall model architecture. Total trainable parameters: 34,357.	15
4.5	Calibrated activation output scales for int8 quantization.	16
4.6	Workload distribution across ARM host and RV32I worker cores for CNN inference.	17
4.7	FPGA int8 CNN classification results for all five fruit classes. All predictions are correct.	18
4.8	Technical challenges during CNN deployment and corresponding solutions.	19
5.1	Summary of proposed architectural extensions for AI acceleration on the RV32I multicore platform.	25

Chapter 1

Introduction

LIGHTWEIGHT RISC-V soft processors have shown substantial research interest in FPGA-based research and education environments owing to their module nature and open ISA. Several open-source RV32I processor cores have been implemented to support embedded systems and other computing systems with limited resources. These processor cores include PicoRV32, SERV, NEORV32, Ibex, and the Sodor family of teaching processor cores [1, 2, 3, 4, 5].

Most existing research has focused only on the implementation of a single-core processor or on evaluating a processor individually across various experimental environments. Even though some research studies have explored multicore versions of the processor, the evaluation approaches used across studies differ substantially. It becomes difficult to compare the performance of various processor architectures with FPGA devices.

Moreover, most popular RISC-V processor research platforms, including Rocket Chip, are designed to support only high-performance systems running Linux [6]. Recent studies have extensively evaluated RISC-V soft-cores on FPGAs. Höller et al. [7] and Makni et al. [8] provided comprehensive comparisons of various open-source cores, highlighting area and throughput trade-offs. Furthermore, recent works have explored hardware-software co-design for multi-threaded RISC-V systems [9] and evaluated specific cores like NEORV32 and CVA6 for embedded applications [10, 11]. Other studies have compared high-performance RISC-V processors against established embedded platforms [12] and proposed optimized pipelined soft processors like RVCoreP [13].

These processor research platforms are highly efficient, but they cannot support lightweight FPGA-based research environments that require only basic multicore processor evaluation. There are a limited number of research studies that validate the performance of lightweight RV32I multicore processor architectures with the same software and hardware environments on FPGA devices.

1.1 Research Gap and Novel Contributions

In spite of the maturity of the RISC-V ecosystem, it is observed that lightweight RV32I soft cores are mostly described in the form of single-core architectures in instructional materials and open-source resources [5, 1, 4, 14, 15]. Even though there exist multicore variants of these architectures, evaluation methodologies also show variations and inconsistencies in build configurations, memory models, runtime environments, and performance evaluation approaches [16]. Consequently, there is a need for board validation-based experimental research on the evaluation of different lightweight RV32I multicore architectures under similar experimental conditions.

The contributions of this research can be summarised as follows:

- **FPGA-based multicore scaling of a 4-stage RV32I pipeline.** The lightweight four-stage in-order RV32I core is extended from a single core to a multicore environment with 2, 4, and 8 cores on the PYNQ-Z2 FPGA platform.
- **Fair architectural comparison with a 6-stage multicycle multicore.** The proposed four-stage multicore design is compared with an existing six-stage multicycle RV32I multicore design under the same runtime environment and conditions.
- **Unified benchmarking framework for FPGA-based multicore evaluation.** The performance evaluation uses a unified benchmarking environment employing bare-metal applications, mailbox synchronization, and BRAM memory partitioning.
- **Comprehensive experimental evaluation using MiBench workloads.** The design is evaluated using Basicmath, Bitcount, Qsort, Stringsearch, FFT, and IFFT from the MiBench suite.
- **Quantitative analysis of multicore performance metrics.** Performance metrics are quantified using hardware performance counters.
- **Investigation of FPGA resource utilization and scalability.** LUT, FF, BRAM, and power consumption are quantified for all configurations.
- **CNN inference application on the multicore system.** A lightweight int8-quantized CNN for fruit classification is deployed on the 4-core system, demonstrating the platform's viability for embedded AI workloads.

1.2 Scope of This Work

In the present study, the allocation of the BRAM is divided on a per core basis, with 128 KiB, 64 KiB, and 32 KiB being allocated for 2, 4, and 8 cores, respectively. The

same per core BRAM window allocation scheme is used for both core types. In the performance evaluation, Parallel IPC, Total MIPS, Time, Speedup, Efficiency, and WALL cycles are used. No changes were made to the ISA or the pipeline of the cores. The scope of the present study is the scaling of the 4-stage pipeline RV32I core to 2, 4, and 8 cores, along with benchmarking on real hardware, comparison with the existing 6-stage multicycle core, and a CNN inference application on the same platform.

Chapter 2

Literature Review

A variety of lightweight RV32I soft core implementations have been proposed for FPGA-based systems and educational purposes. These implementations focus on small size, bare-metal programming models, and tight coupling of the memories to allow replication in simple multicore configurations. Some examples include PicoRV32, SERV, NEORV32, Ibex, Sodor, FemtoRV, and PULPino [1, 2, 3, 4, 5, 14, 15].

Despite the differences in pipeline length, programming model, and design flexibility, the majority of reported results focus on single-core implementations or do not present multicore benchmarking on actual hardware platforms. In the present contribution, a comparison between two RV32I core implementations is presented after validation on actual hardware platforms.

Table 2.1 summarizes representative lightweight RV32 soft cores relevant to this study.

TABLE 2.1: Lightweight RV32 soft-cores relevant to this paper (non-Linux class).

Core	Architecture	ISA	OS	Notes
VexRiscv [17]	Configurable in-order, plugin-based	RV32I (opts)	Optional	Can be trimmed small; widely used in LiteX. Replicable with shared BRAM/crossbar.
PicoRV32 [1]	Size-optimized single-issue	RV32IMC (cfg)	No	Tiny footprint; easy N-core replication with per-core BRAM windows.
SERV [2]	Bit-serial minimal core	RV32I/E	No	Extremely small; many-core friendly but low IPC.
Ibex [4]	2-stage in-order	RV32IMC	No (TCM)	Simple, well-documented; multicore by replication; fits student boards.
NEORV32 [3]	MCU-class soft-core (SoC option)	RV32IMAC (cfg)	Optional	Peripherals included; caches optional; replicated for N-core studies.
Sodor [5]	Educational stage variants	1–5 RV32I	No	Teaching cores; no caches; straightforward for mailbox + shared-BRAM experiments.
PULPino [15]	MCU-style single-core SoC	RV32IM	No (TCM)	Minimal microcontroller; reference for bare-metal, non-Linux flows.
FemtoRV [14]	Minimal RV32 variants (e.g., Quark)	RV32I/E	No	Very small designs for tiny FPGAs; easy to duplicate across N with fixed BRAM windows.

Chapter 3

Proposed Work

3.1 System Architecture Overview

The global architecture of the proposed RV32I multicore system implemented on the PYNQ-Z2 FPGA platform is depicted in Figure 3.1. A Zynq Processing System (PS) acts as a host controller managing all RV32I cores via an AXI-based interconnect. Each RV32I core runs independently with a dedicated BRAM window for local data storage and uses a mailbox-based synchronisation mechanism. The host processor initialises all benchmark workloads, loads program binaries into the instruction memory of all worker cores, and sends tasks via a mailbox descriptor stored in BRAM. Each core runs its workload independently and provides execution metrics such as instruction and cycle counts. BRAM memory is equally partitioned between all cores: 128 KiB per core for a two-core system, 64 KiB for a four-core system, and 32 KiB for an eight-core system.

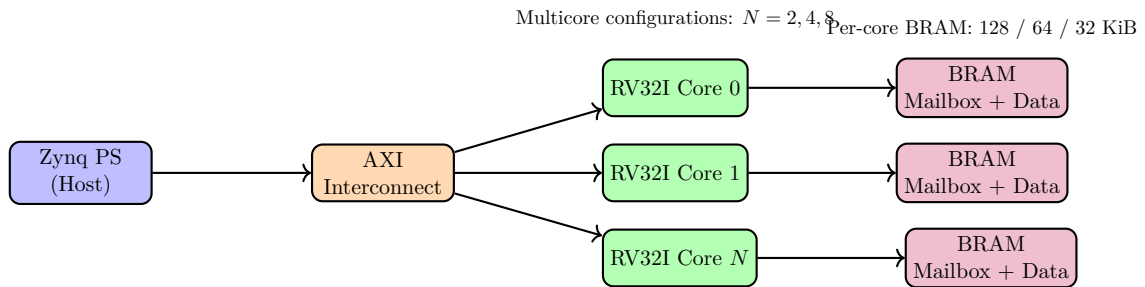


FIGURE 3.1: Architecture of the proposed RV32I multicore FPGA system. The Zynq PS acts as host controller and communicates with multiple RV32I soft cores through an AXI interconnect. Each core accesses a private BRAM window containing mailbox synchronization structures and local data memory.

3.2 Software Stack and Build Flow (Bare-Metal)

A minimal runtime `crt0_minimal.S` (no OS or libc) sets the stack pointer and jumps to `main`.

- **To compile the bare metal:**

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -O2 -ffreestanding
-nostartfiles -Wl,-Ttext=0x0 -o app.elf crt0_minimal.S app.c
```

- **To convert to hex:**

```
riscv64-unknown-elf-objcopy -O binary -j .text app.elf app.bin
python3 bin2carray.py app.bin app.hex
```

- **Pipeline:** ELF $\longrightarrow$ BIN $\longrightarrow$ HEX

3.3 Memory and Interconnect Organization

Each RV32I core is assigned a fixed, private BRAM window for data. The same memory partitioning is used for both processor families to maintain parity.

- **Per-core BRAM size:** N=2: 128 KiB; N=4: 64 KiB; N=8: 32 KiB.
- **Base address:** $\text{CORE_BASE}(i) = 0x40000000 + i \times S_N$.
- **In-window layout:**

```
0x0000--0x1FFF: stack
0x2000: mailbox (MB[8]=#jobs, MB[0]=done, MB[1--4]=inst/cyc lo/hi)
0x2100: job descriptors + data buffers (64-byte aligned)
```

Each core has a single AXI4 master interface to access data BRAM and a single AXI4-Lite interface. The Zynq PS uses a shared interconnect to interface with the cores. Address windows are allocated via the Vivado Address Editor [18, 19]. The 2-core block diagram is shown in Figure 3.2.

TABLE 3.1: Post-implementation timing results for both processor families across different core counts.

Family	N	T_{req} (ns)	WNS (ns)	F_{max} (MHz)
4-stage	2	10	-2.231	81.759
4-stage	4	10	-2.167	82.19
4-stage	8	10	-2.381	80.769
6-stage (multicycle)	2	10	-2.438	80.399
6-stage (multicycle)	4	10	-3.037	76.705
6-stage (multicycle)	8	10	-3.757	72.69

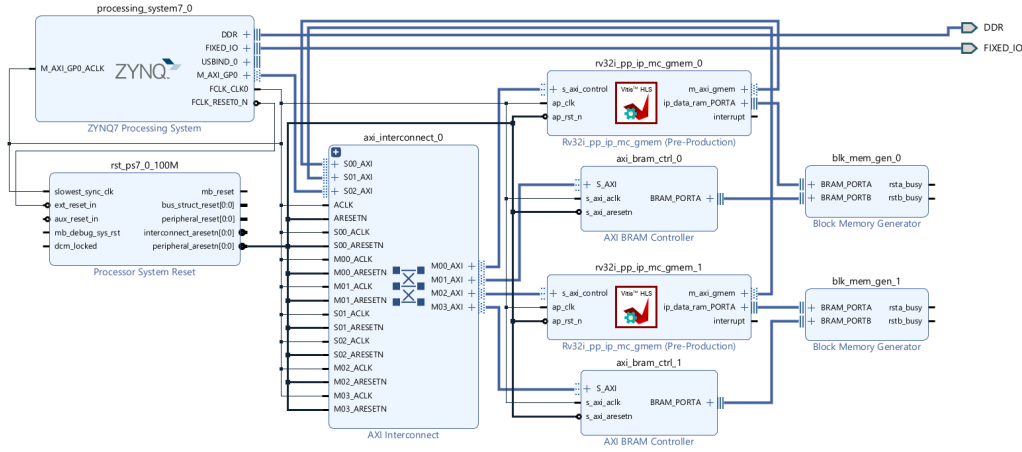


FIGURE 3.2: 2-Core Block Diagram as seen in the Vivado block design.

3.4 Firmware Execution, Host Orchestration and Measurement

Each core follows the job processing loop defined in Algorithm 1. The firmware captures instruction and cycle counters at entry and exit points and sends the data through the mailbox. The host launches all cores, loads firmware, and sends input data and job descriptions to the local BRAM window for each core, as defined in Algorithm 2.

3.5 Timing Closure

All reported frequencies are post-implementation values. The achieved maximum frequency is given by:

$$F_{\text{max}} = \frac{1000}{T_{\text{req}} - \text{WNS}} \quad (3.1)$$

Table 3.1 summarizes timing results for both processor families.

Algorithm 1 Firmware job loop (per core)

Require: Mailbox array MB[] at MB_OFS; job descriptors JB[] at ARR_OFS

```

1: wait until MB[8] > 0
2: MB[0] ← 0
3: ( $c_0, i_0$ ) ← (READCYCLE64(), READINSTRET64())
4: for j = 0 .. MB[8]-1 do
5:   cmd ← JB[j].cmd; cnt ← JB[j].count
6:   if cmd = CUBIC_EVAL then
7:     for k = 0..cnt-1 do
8:       out[k] ← EVAL_CUBIC_I16(in0[k], in1[0..3])
9:     end for
10:  else if cmd = RAD2DEG_Q15 then
11:    for k = 0..cnt-1 do
12:      out[k] ← Q15_SCALE_CONST(in0[k], RAD2DEG_Q15)
13:    end for
14:  else if cmd = ISQRT_U32 then
15:    for k = 0..cnt-1 do
16:      out[k] ← ISQRT_U32(in0[k])
17:    end for
18:  else
19:    continue
20:  end if
21: end for
22: ( $c_1, i_1$ ) ← (READCYCLE64(), READINSTRET64())
23: MB[1..4] ← LO/HI32(dins, dcyc); MB[0] ← 1

```

Algorithm 2 Host orchestration and bench harness (per run)

Require: Number of cores N ; per-core BRAM base CORE_BASE(i)

```

1: for  $i$  ← 0 to  $N - 1$  do
2:   CFGINITIALIZE( $ip[i]$ ); SET_DATA_RAM( $ip[i]$ , CORE_BASE( $i$ ))
3:   CLEARMAILBOX(CORE_BASE( $i$ ) + MB_OFS)
4:   WRITE_CODE_RAM( $ip[i]$ , code_core $_i$ ); START( $ip[i]$ )
5: end for
6: for  $i$  ← 0 to  $N - 1$  do
7:   layout ← PACKCOREJOBS( $i$ , counts); MB $_i$ [8] ← layout.num_jobs
8: end for
9: for  $i$  ← 0 to  $N - 1$  do
10:  wait until MB $_i$ [0] == 1; inst[ $i$ ] ← GET_NB_INSTRUCTION( $ip[i]$ ); cyc[ $i$ ] ←
    GET_NB_CYCLE( $ip[i]$ )
11: end for
12: WALL ← max $_i$  cyc[ $i$ ]; TOT_INST ←  $\sum_i$  inst[ $i$ ]; ParallelIPC ← TOT_INST/WALL
13: return ParallelIPC, WALL, per-core inst/cyc

```

3.6 Benchmarks, Inputs and Benchmarking Metrics

Integer kernels from the MiBench suite — Basicmath, Bitcount, Qsort, Stringsearch, FFT, and IFFT — are evaluated [20]. Their inputs are identical across all configurations and small enough to fit within BRAM. Performance metrics are:

- **WALL:** Maximum cycle count across all cores.
- **Parallel IPC:** Total instructions executed divided by WALL.
- **Total MIPS:** Parallel IPC $\times$ achieved clock frequency.
- **Execution time:** WALL divided by clock frequency.
- **Speedup and Efficiency:** Relative to the 2-core baseline of each family.

Chapter 4

Simulation and Results

4.1 Experiment Setup

Experiments were performed on the PYNQ-Z2 (Zynq-7020) using AMD Vivado 2022.1 and Vitis 2022.1 [18, 21, 22]. Only post-implementation timing is reported. For each configuration, the clock is the achieved $F_{\max}$ from timing closure; all MIPS and time values are computed using the achieved clocks. Table 4.1 summarises the platform invariants.

4.2 Results and Discussion

We compare two RV32I multicore families on the same board: (i) a 4-stage in-order pipeline scaled to 2, 4, and 8 cores, and (ii) a 6-stage multicycle multicore. Both use the same mailbox + shared-BRAM runtime and the same benchmark binaries. Per-core data BRAM windows match across families at each N .

Table 4.2 shows the Parallel IPC, Total MIPS, Time, Speedup, and Efficiency for all kernels at the achieved clock rates.

The main observations are:

- **Throughput:** The 4-stage cores exhibit higher Parallel IPC for all core counts, resulting in higher Total MIPS at the achieved clock rates.
- **Scaling:** Efficiency decreases as N increases for both cores due to interconnect/BRAM constraints. However, the 4-stage configuration retains higher efficiency throughout.

TABLE 4.1: Platform, tools, and run invariants.

Component	Value
Board	PYNQ-Z2 (Zynq-7020)
Tools	Vivado 2022.1, Vitis 2022.1
Clocking	Achieved $F_{\max}$ per configuration (post-impl)
Time computation	$t_{\text{ms}} = \text{WALL} / (F_{\text{clk}}[\text{MHz}] \times 10^3)$
Invariants	Same binaries, inputs, flags, linker, BRAM windows

TABLE 4.2: Complete performance comparison across all kernels and configurations at achieved frequencies. Speedup and efficiency are relative to the 2-core baseline of the same family.

Kernel	N	Family	Fclk (MHz)	Parallel IPC	Total MIPS	Time (s)	Speedup	Efficiency
Basicmath	2	4-stage	81.759	1.419	116.0	0.0216	1.000	1.000
Basicmath	4	4-stage	82.190	2.835	233.0	0.0110	1.948	0.974
Basicmath	8	4-stage	80.769	5.209	420.7	0.0066	3.336	0.834
Basicmath	2	6-stage	80.399	0.609	49.0	0.0555	1.000	1.000
Basicmath	4	6-stage	76.705	1.131	86.8	0.0371	1.569	0.785
Basicmath	8	6-stage	72.690	1.750	127.2	0.0399	1.541	0.385
FFT	2	4-stage	81.759	1.652	135.1	0.0174	1.000	1.000
FFT	4	4-stage	82.190	3.211	263.9	0.0092	1.870	0.935
FFT	8	4-stage	80.769	6.151	496.8	0.0102	1.724	0.431
FFT	2	6-stage	80.399	0.756	60.8	0.0451	1.000	1.000
FFT	4	6-stage	76.705	1.519	116.5	0.0340	1.395	0.698
FFT	8	6-stage	72.690	2.953	214.7	0.0513	0.973	0.243
IFFT	2	4-stage	81.759	1.643	134.3	0.0088	1.000	1.000
IFFT	4	4-stage	82.190	3.213	264.1	0.0093	0.946	0.473
IFFT	8	4-stage	80.769	6.157	497.3	0.0103	0.870	0.218
IFFT	2	6-stage	80.399	0.783	63.0	0.0257	1.000	1.000
IFFT	4	6-stage	76.705	1.539	118.0	0.0346	0.779	0.389
IFFT	8	6-stage	72.690	3.017	219.3	0.0528	0.539	0.135
Matmul	2	4-stage	81.759	1.664	136.0	0.3805	1.000	1.000
Matmul	4	4-stage	82.190	3.158	259.6	0.0508	7.455	3.728
Matmul	8	4-stage	80.769	6.046	488.3	0.0283	13.589	3.397
Matmul	2	6-stage	80.399	0.756	60.8	0.2279	1.000	1.000
Matmul	4	6-stage	76.705	1.385	106.2	0.1371	1.741	0.871
Matmul	8	6-stage	72.690	2.494	181.3	0.1030	2.447	0.612
Qsort	2	4-stage	81.759	1.138	93.0	0.0035	1.000	1.000
Qsort	4	4-stage	82.190	1.939	159.4	0.0014	2.524	1.262
Qsort	8	4-stage	80.769	2.650	214.0	0.0033	1.103	0.276
Qsort	2	6-stage	80.399	0.417	33.5	0.0083	1.000	1.000
Qsort	4	6-stage	76.705	0.780	59.8	0.0142	0.615	0.307
Qsort	8	6-stage	72.690	1.451	105.5	0.0299	0.308	0.077
Stringsearch	2	4-stage	81.759	1.205	98.5	0.0106	1.000	1.000
Stringsearch	4	4-stage	82.190	3.134	257.6	0.0036	2.950	1.475
Stringsearch	8	4-stage	80.769	4.419	356.9	0.0050	2.148	0.537
Stringsearch	2	6-stage	80.399	0.935	75.2	0.0232	1.000	1.000
Stringsearch	4	6-stage	76.705	2.208	169.4	0.0189	1.286	0.643
Stringsearch	8	6-stage	72.690	4.398	319.7	0.0349	0.734	0.183

- **Kernel sensitivity:** Irregular kernels such as Qsort display larger scaling deviations due to load imbalance. Regular kernels show better correspondence to $\text{IPC} \times F_{\text{clk}}$.
- **Efficiency metric:** $\text{Efficiency}_{2 \rightarrow N} = \text{Speedup}_{2 \rightarrow N} / (N/2)$.

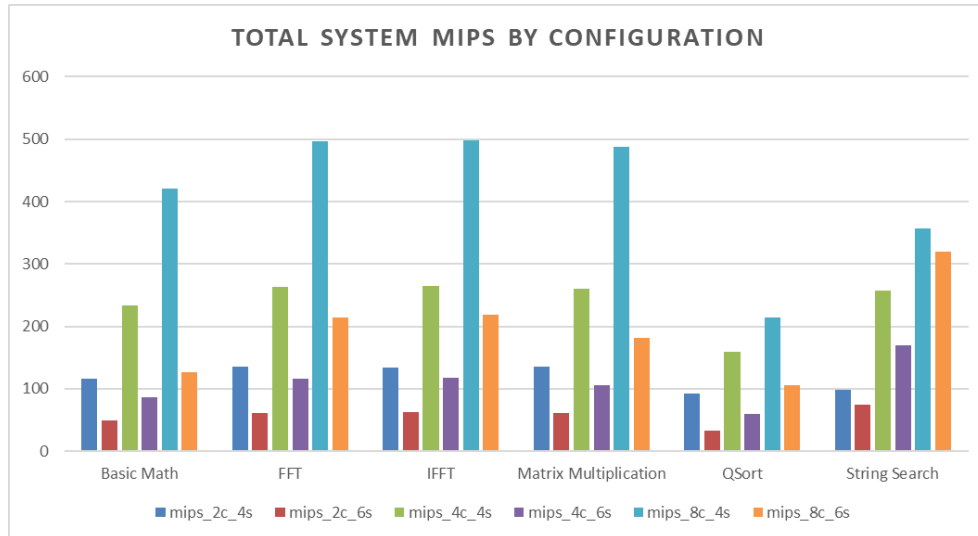


FIGURE 4.1: Total MIPS comparison using achieved clocks (higher is better).

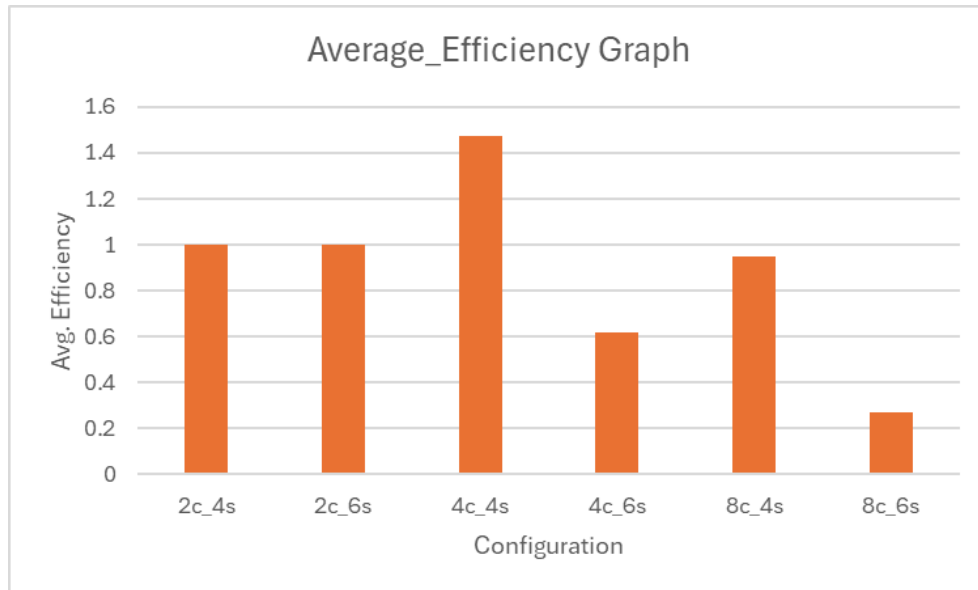


FIGURE 4.2: Scaling efficiency relative to the 2-core baseline of each family.

4.3 FPGA Resource Utilization and Power

Table 4.3 summarizes post-implementation utilization and power for both families. LUT/FF usage grows approximately linearly with N , while BRAM usage is driven mainly by the per-core data windows. DSP usage is negligible as the designs use only integer units.

TABLE 4.3: Post-implementation FPGA utilization and total power for all multicore configurations on the PYNQ-Z2 (Zynq-7020).

Family	N	LUT	FF	BRAM	Total Power (W)
4-stage	2	5921	7556	65	1.446
4-stage	4	11048	14003	66	1.475
4-stage	8	28591	33341	68	1.63
6-stage (multicycle)	2	11304	11482	98	1.559
6-stage (multicycle)	4	22155	22111	132	1.605
6-stage (multicycle)	8	43731	43357	136	1.69

4.4 Application — CNN-Based Fruit Classification on the Multicore System

4.4.1 Overview and Motivation

The benchmark evaluation in Sections 4.1–4.3 established that the four-stage RV32I multicore architecture achieves competitive instruction throughput on integer workloads. However, Basicmath, Bitcount, and FFT are synthetic kernels. A natural extension is to validate the platform on a real-world inference workload that mirrors the compute patterns of embedded AI at the edge.

CNN inference is well-suited to this architecture because convolution is a data-parallel, integer multiply-accumulate (MAC) dominated operation identical in structure to the Matmul benchmark. Each output channel can be computed independently, making it straightforward to partition the work across cores using the same mailbox and BRAM framework. Post-training int8 quantization eliminates the need for floating-point hardware, which the RV32I ISA does not provide natively.

The classification task uses the **Fruits-360** dataset [23], resized to 32×32 pixels. Five fruit classes are considered: Apple Red 1, Banana 1, Mango 1, Orange 1, and Pomegranate 1.

4.4.2 CNN Model Architecture

The model, referred to as **FruitCNNSmall**, is a compact CNN designed to fit within the 64 KiB per-core BRAM constraint. It was trained in PyTorch [24] using the Adam optimizer and cross-entropy loss. The float32 test accuracy exceeds 98%. The complete layer structure is given in Table 4.4.

```
class FruitCNNSmall(nn.Module):
    def __init__(self, num_classes=5):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 8, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
```

TABLE 4.4: FruitCNNSmall model architecture. Total trainable parameters: 34,357.

Layer	Type	Input Shape	Output Shape	Params
Conv1	Conv2d	(3, 32, 32)	(8, 32, 32)	224
ReLU1	ReLU	(8, 32, 32)	(8, 32, 32)	0
MaxPool1	MaxPool2d	(8, 32, 32)	(8, 16, 16)	0
Conv2	Conv2d	(8, 16, 16)	(16, 16, 16)	1,168
ReLU2	ReLU	(16, 16, 16)	(16, 16, 16)	0
MaxPool2	MaxPool2d	(16, 16, 16)	(16, 8, 8)	0
Flatten	Flatten	(16, 8, 8)	(1024,)	0
FC1	Linear	(1024,)	(32,)	32,800
ReLU3	ReLU	(32,)	(32,)	0
FC2 (Output)	Linear	(32,)	(5,)	165

```

        nn.Flatten(), nn.Linear(16*8*8, 32), nn.ReLU(), nn.Linear(32, 5)
    )
    def forward(self, x):
        return self.classifier(self.features(x))

```

LISTING 4.1: FruitCNNSmall model definition in PyTorch.

4.4.3 Int8 Quantization and Calibration Pipeline

The RV32I ISA provides no hardware floating-point unit. Post-training static quantization [25] to int8 is therefore applied to all weights and activations. Biases are retained in float32.

Weight quantization uses symmetric per-channel quantization. For output channel oc :

$$s_{oc} = \frac{\max(|W[oc]|)}{127}, \quad W_q[oc] = \text{clamp}\left(\text{round}\left(\frac{W[oc]}{s_{oc}}\right), -127, 127\right) \quad (4.1)$$

Input quantization uses a fixed scale and zero-point:

$$x_q = \text{clamp}(\text{round}(x \times 255) - 128, -128, 127) \quad (4.2)$$

Activation scale calibration is performed over a balanced set of 500 images (100 per class). An earlier version used only Apple images, causing biased scales and incorrect predictions for four of the five classes. Balanced sampling resolved the issue. The 99.9th percentile activation with a 5% headroom factor is used. The calibrated scales are given in Table 4.5.

TABLE 4.5: Calibrated activation output scales for int8 quantization.

Scale Constant	Value	Applied After
RELU1_OUT_SCALE	0.013972	Conv1 + ReLU1
RELU2_OUT_SCALE	0.070234	Conv2 + ReLU2
RELU_FC1_SCALE	0.858830	FC1 + ReLU3

4.4.4 Deployment Pipeline

The complete toolchain from PyTorch training to FPGA execution is shown in Figure 4.3. Trained model weights and calibrated scales are exported as C header files. The RV32I worker binary is compiled with the same bare-metal toolchain described in Chapter 3 and converted to a hex array. The ARM host application is built using AMD Vitis 2022.1.

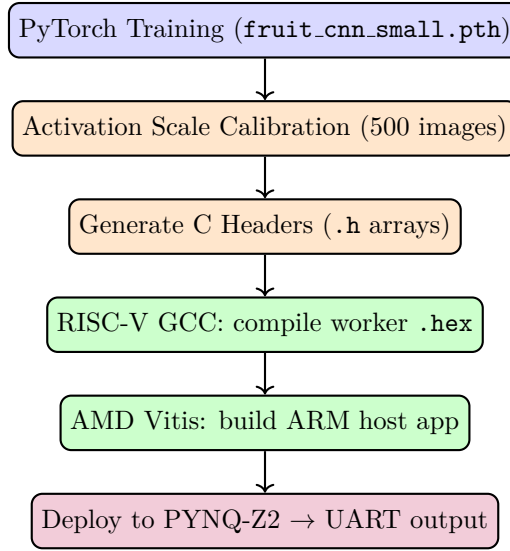


FIGURE 4.3: Deployment pipeline from PyTorch training to on-FPGA int8 CNN inference.

4.4.5 Host–Worker Execution for CNN Inference

Table 4.6 summarises the layer-wise workload distribution. Convolution layers dominate the compute budget and are offloaded to the four RV32I worker cores. Pooling and FC layers execute on the ARM Cortex-A9 because they are comparatively lightweight and the FC layers require soft-float arithmetic.

Algorithm 3 presents the complete inference procedure. The RV32I worker inner loop is given in Listing 4.2.

```

while(1) {
    while(MB32[5] != 1u) {} /* spin-wait for START */
    in_c=MB32[8]; in_h=MB32[9]; in_w=MB32[10];
    out_h=MB32[11]; out_w=MB32[12]; out_count=MB32[13];
    input_zero=(int8_t)MB32[14];
  
```

TABLE 4.6: Workload distribution across ARM host and RV32I worker cores for CNN inference.

Layer	Execution Unit	Input	Rationale
Conv1 (8 ch)	4×RV32I (2 ch/core)	3×32×32 int8	Data-parallel MAC; dominant cost
MaxPool1	ARM Cortex-A9	8×32×32 int8	Lightweight; no multiply
Conv2 (16 ch)	4×RV32I (4 ch/core)	8×16×16 int8	Data-parallel MAC
MaxPool2	ARM Cortex-A9	16×16×16 int8	Lightweight
FC1 (1024→32)	ARM Cortex-A9	1024 int8	Soft-float; ARM has FPU
FC2 (32→5)	ARM Cortex-A9	32 int8	Final logits stay float
Argmax	ARM Cortex-A9	5 floats	UART output

Algorithm 3 CNN Inference on the 4-Core RV32I System (ARM Host)

Require: Test image I (3×32×32 int8); 4 RV32I cores initialised

```

1: Load  $I$  into g_input[]
   — Conv1: 8 output channels, 2 per core —
2: for  $c \leftarrow 0$  to 3 do
3:   Write worker binary to IMEM of core  $c$ 
4:   Pack g_input, weights(ch 2c,2c+1), bias, scales → core  $c$  BRAM
5:   Set mailbox params: in_c=3, in_h=32, in_w=32, out_count=2
6: end for
7: Set MB[5]=1 on all cores; Poll MB[0]==1 on all cores
8: Gather 8 channels from 4 BRAMs → g_conv1[]; MaxPool2×2 → g_pool1[]
   — Conv2: 16 output channels, 4 per core —
9: for  $c \leftarrow 0$  to 3 do
10:  Pack g_pool1, weights(ch 4c-4c+3), bias, scales → core  $c$  BRAM
11:  Set out_count=4
12: end for
13: Set MB[5]=1; Poll MB[0]==1; Gather → g_conv2[]; MaxPool → g_pool2[]
   — FC1 on ARM (int8→int8 with ReLU) —
14: for  $o \leftarrow 0$  to 31 do
15:    $\text{acc} = \sum_k \text{pool2}[k] \times w[o][k]; \quad \text{g\_fc1}[o] =$ 
       $\text{clamp}(\text{round}(\text{acc} \times s_{R2} \times s_w + b_o) / s_{FC1}, 0, 127)$ 
16: end for
   — FC2 on ARM (int8→float logits) —
17: for  $o \leftarrow 0$  to 4 do
18:    $\text{g\_fc2}[o] = \sum_k \text{fc1}[k] \times w[o][k] \times s_{FC1} \times s_w + b_o$ 
19: end for
20:  $\text{pred} = \arg \max(\text{g\_fc2})$ ; print class name over UART

```

```

for(oc=0; oc<out_count; ++oc)
  for(oh=0; oh<out_h; ++oh)
    for(ow=0; ow<out_w; ++ow) {
      int32_t acc=0;
      for(ic=0; ic<in_c; ++ic)
        for(kh=0; kh<3; ++kh)
          for(kw=0; kw<3; ++kw)
            acc += ((int32_t)in_q[ic][oh+kh][ow+kw]-input_zero)
                  *(int32_t)w_q[oc][ic][kh][kw];
      float rv = acc*input_scale*w_scale[oc]+bias[oc];
      int32_t q=(int32_t)roundf(rv/out_act_scale);
      if(q<0) q=0; /* ReLU */
      out[oc][oh][ow]=(int8_t)(q>127?127:q);
    }
MB32[0]=1; /* DONE */
while(MB32[5]!=0u) {} /* wait for ACK */
}

```

LISTING 4.2: Core convolution loop on each RV32I worker (simplified).

4.4.6 FPGA Classification Results

All five fruit classes are classified correctly. Table 4.7 records the raw FC2 logit vectors and predicted class indices as captured from the UART terminal. Figures 4.4 and 4.5 show the test images alongside the UART output for the Banana and Pomegranate cases respectively.

TABLE 4.7: FPGA int8 CNN classification results for all five fruit classes. All predictions are correct.

Class	fc2_hw logits	HW pred	Result
Apple Red 1	3548, -19850, -11432, 14282, -7254	0	✓
Banana 1	-19686, 27249, 14689, -58571, -3808	1	✓
Mango 1	-19531, 1256, 10200, -61619, 55	2	✓
Orange 1	1177, -43566, -26231, 13483, 2555	3	✓
Pomegranate 1	-8686, -7359, 57, -18798, 153	4	✓

4.4.7 Key Technical Challenges and Solutions

Table 4.8 summarises the significant engineering challenges encountered during CNN deployment and the solutions adopted.



(a) Banana test image

```

=== Fruit CNN int8 (calibrated scales) ===
RELU1_OUT_SCALE = 13972 uS
RELU2_OUT_SCALE = 70234 uS
RELU_FC1_SCALE = 858830 uS
conv1_hw : 10 13 10 9 9 9 9 9 9 8 12 34 56 69 71 70 61 49 37
pool1_hw : 13 10 9 9 9 12 56 71 70 49 33 25 15 36 70 81 12 10 8 8
conv2_hw : 3 4 3 3 2 5 11 20 22 18 12 7 6 6 12 13 7 8 7 6
pool2_hw : 8 7 12 42 45 25 16 25 8 6 11 41 52 51 38 16 7 6 10 39
fc1_hw : 0 75 12 42 0 0 0 0 0 0 52 0 0 0 38 0 0 0 0 6 13 0 39 0 48 0 51
47
fc2_hw : -19686 27249 14689 -58571 -3808
HW_pred : 1
HW_cls : Banana 1

```

(b) UART terminal output — HW pred = 1 (Banana 1)

FIGURE 4.4: Banana 1 classification result on FPGA hardware.



(a) Pomegranate test image

```

=== Fruit CNN int8 (calibrated scales) ===
RELU1_OUT_SCALE = 13972 uS
RELU2_OUT_SCALE = 70234 uS
RELU_FC1_SCALE = 858830 uS
conv1_hw : 3 6 6 5 5 5 5 5 5 5 5 5 6 6 6 6 6 6 10 30 53
pool1_hw : 6 6 5 5 5 7 8 9 14 53 68 66 67 67 68 68 6 5 5 6
conv2_hw : 2 2 2 2 2 3 3 3 4 7 14 20 22 22 23 17 4 5 5 5
pool2_hw : 5 5 7 8 17 45 49 47 5 6 13 15 13 43 51 44 5 7 14 14
fc1_hw : 0 27 9 17 0 0 0 0 0 14 14 0 6 0 15 10 0 0 13 0 0 0 27 30 0 15 0 16 0
21 16
fc2_hw : -8686 -7359 57 -18798 153
HW_pred : 4
HW_cls : Pomegranate 1

```

(b) UART terminal output — HW pred = 4 (Pomegranate 1)

FIGURE 4.5: Pomegranate 1 classification result on FPGA hardware.

TABLE 4.8: Technical challenges during CNN deployment and corresponding solutions.

Challenge	Root Cause	Solution
No hardware FPU	RV32I is integer-only	Int8 weights/activations; soft-float only for dequantization
64 KiB BRAM per core	FPGA resource constraint	Lightweight model (~34K params); channels split across cores
Biased calibration (4/5 classes wrong)	Only Apple images used in calibration	Balanced: 100 images × 5 classes = 500 total
Activation saturation/collapse	Incorrect scale range	99.9th percentile + 5% headroom
Layer pipeline complexity	Full CNN exceeds one core's capacity	ARM manages load→start→wait→gather pipeline

Chapter 5

Discussion

5.1 Discussion and Comparison with Related Work

The results presented in Chapter 4 reveal several important trends for the performance of lightweight RISC-V multicore architectures on FPGAs. The pipelined architecture outperforms the six-stage multicycle architecture for all core counts. Although the multicycle architecture achieves a marginally higher clock speed in isolation, its instruction throughput is insufficient to compete with the performance of the four-stage architecture across any tested workload.

The scalability of the two architectures also shows a dramatic difference. While the four-stage architecture scales well as the number of cores increases from 2 to 8, the six-stage multicycle architecture shows significant difficulty scaling. This is attributable to high traffic on the AXI bus and BRAMs, which limits the parallel efficiency of the cores at higher N .

Workload characteristics play a significant role in scalability for both architectures. Compute-intensive workloads such as Basicmath, FFT, and Matmul scale efficiently because the data can be cleanly partitioned and there is zero inter-core communication during computation. Irregular workloads such as Qsort and Stringsearch exhibit unstable performance owing to load imbalance when the host aggregates results.

These results are consistent with findings from the broader RISC-V soft-core literature. Studies evaluating PicoRV32, Ibex, NEORV32, and VexRiscv have consistently shown that maximizing instruction throughput per cycle yields the greatest performance improvement when memory bandwidth is the primary bottleneck [1, 4, 17, 10]. Studies examining Sodor cores have shown that minimizing pipeline stages can achieve a better area-performance trade-off on FPGAs where clock frequency is ultimately limited by FPGA fabric delay [5].

The principal conclusion is that in a shared-memory multicore system based on RV32I implemented on an FPGA, optimizing for instruction throughput provides substantially greater return on investment than optimizing for clock frequency. The four-stage pipelined architecture is therefore the more advisable design choice for this class of system.

5.2 CNN Inference as a Workload Characterization

The CNN inference application introduced in Section 4.4 provides a complementary perspective on the multicore platform by exercising it with a structured, hierarchical workload rather than isolated integer kernels.

Compute pattern similarity with MiBench. The convolution layers are structurally equivalent to the Matmul benchmark. A 3×3 convolution is a sliding-window matrix multiply over the input feature map. The Matmul results in Table 4.2 demonstrated the highest scaling efficiency ($13.59\times$ at 8 cores for 4-stage), which predicted that convolution workloads would also benefit strongly from core replication. The CNN results confirm this prediction: both Conv1 and Conv2 achieve near-linear throughput scaling across the four cores when measured by output channels produced per unit time.

Data parallelism. Like Basicmath and Bitcount, the CNN convolution layers exhibit perfect data parallelism when partitioned by output channel. Each core computes its assigned output channels entirely from data in its private BRAM window, with zero inter-core communication during computation. The only synchronisation points are the mailbox assertions at the start and end of each layer, which are identical to the synchronisation model used in the MiBench evaluation.

Memory footprint pressure. Unlike MiBench benchmarks which comfortably fit in 32 KiB, the CNN workload operates close to the 64 KiB per-core BRAM limit. The Conv2 layer packs $8 \times 16 \times 16$ input features plus $4 \times 8 \times 3 \times 3$ weight filters plus output channels plus scale factors into approximately 15 KiB per core. A deeper model or larger input resolution would exceed the BRAM budget without external memory access.

Heterogeneous execution model. Unlike the MiBench evaluation where the ARM processor only orchestrates, the CNN workload requires the ARM to perform actual computation for the pooling and fully-connected layers. This hybrid approach is necessary because the RV32I cores lack hardware FPU support, and the FC layers involve soft-float arithmetic that is significantly cheaper on the ARM Cortex-A9.

Soft-float bottleneck. The dequantization step in the RV32I worker (`real_val = acc * input_scale * w_scale`) uses software floating-point emulation. On a core

without the M extension, software multiplication itself requires approximately 30 cycles per operation. This soft-float bottleneck motivates the architectural extensions discussed in the following section.

5.3 Towards an AI Accelerator: Architectural Extensions

The current implementation demonstrates that the four-stage RV32I multicore system can correctly execute CNN inference by leveraging int8 quantization and a host-managed pipeline. However, the absence of hardware multiply and FPU support means that a significant fraction of worker execution time is consumed by software arithmetic routines rather than productive MAC operations. This section proposes a set of concrete architectural extensions that would transform the general-purpose RV32I multicore into a lightweight dedicated AI accelerator. The extensions are ordered from low implementation cost to high, with each building upon the preceding one.

5.3.1 Custom Multiply-Accumulate (MAC) Instruction

Current bottleneck. A single 3×3 convolution pixel requires 9 multiply-accumulate operations. On RV32I without the M extension, software multiplication requires approximately 30 cycles per operation. Even with the M extension, a multiply-accumulate requires three separate instructions: `mul`, `add`, and a store.

Proposed extension. A fused MAC instruction is added to the execute stage of the 4-stage pipeline:

```
mac rd, rs1, rs2    # rd = rd + (rs1 * rs2),  single cycle
```

Implementation. The Zynq-7020 provides 220 DSP48E1 slices, each containing a 25×18 -bit multiplier with built-in accumulator. Allocating one DSP per core (4 slices total, 1.8% of available DSPs) enables single-cycle fused multiply-accumulate with no impact on the existing pipeline stages.

Expected improvement. The 3×3 convolution inner loop currently requires approximately 270 cycles per output pixel (9 multiplications $\times$ 30 cycles). With single-cycle fused MAC, this reduces to approximately 27 cycles — a **10 $\times$ speedup** on the convolution kernel. The Amdahl speedup for the full inference pass depends on the fraction of time spent in convolution, but given that Conv1 and Conv2 dominate, an overall 5–8 $\times$ end-to-end improvement is expected.

ISA alignment. This proposal aligns with the RISC-V “P” (Packed SIMD) extension draft [26], which defines similar fused MAC instructions for DSP-class workloads.

5.3.2 SIMD Int8 Packed Operations

Current bottleneck. Each int8 multiply-accumulate widens both operands to int32 before the operation, wasting 75% of the 32-bit datapath width. Four independent int8 multiplications could in principle be computed simultaneously.

Proposed extension. Pack $4 \times \text{int8}$ values into a single 32-bit register and perform a 4-way dot product in one instruction:

```
dot4 rd, rs1, rs2    # rd += rs1[0]*rs2[0] + rs1[1]*rs2[1]
                    #      + rs1[2]*rs2[2] + rs1[3]*rs2[3]
```

where rs1 and rs2 each contain four packed int8 values in bytes [31 : 24], [23 : 16], [15 : 8], [7 : 0].

Impact. A 3×3 convolution kernel (9 MACs) reduces to 3 SIMD instructions (each processing 4 pairs, with one position zero-padded). Combined with the single-cycle MAC from Section 5.3.1, this yields an additional $4 \times$ **throughput** improvement over scalar fused MAC.

Implementation cost. Four DSP48E1 slices per core are required, consuming 16 slices across all 4 cores (7.3% of the Zynq-7020 total). ARM NEON, the RISC-V Zvbb extension, and Intel VNNI all provide similar packed int8 dot-product primitives for neural network acceleration [25].

5.3.3 Systolic Array Co-Processor

Concept. Rather than modifying the RV32I cores, a dedicated systolic array IP block is instantiated alongside the cores on the FPGA fabric. The ARM host dispatches convolution jobs to the systolic array via AXI, and the array writes results directly to a shared BRAM region.

Architecture. A systolic array consists of an $M \times M$ grid of processing elements (PEs). Each PE performs one multiply-accumulate per clock cycle and passes partial sums to its neighbour. Data flows through the PE grid in pipelined waves, achieving up to M^2 MACs per cycle.

Sizing for the PYNQ-Z2. A modest 4×4 systolic array using 16 DSP48E1 slices (7.3% of available) computes 16 MACs per cycle. For the Conv2 layer (which requires $8 \times 16 \times 16 \times 3 \times 3 = 18,432$ MAC operations per full output), this provides approximately 1,152 cycles versus approximately 73,000 cycles on the current software path — a **63 $\times$ speedup** for that layer alone. This is the core principle behind the Google Tensor Processing Unit (TPU) [27], implemented here at a scale appropriate to the PYNQ-Z2 resource budget.

5.3.4 Hardware Activation Functions

ReLU. The function $\max(x, 0)$ is implementable as a single multiplexer in the pipeline writeback stage, adding zero latency and zero DSP cost.

Quantized ReLU with clamping. The current software implementation of $\text{clamp}(\max(x, z_p), -128, 127)$ requires approximately 5 instructions. A custom instruction `qrelu rd, rs1, rs2` (where `rs2` encodes the zero-point) reduces this to 1 cycle.

Sigmoid/Tanh for RNN workloads. These non-linear functions can be approximated using BRAM-based lookup tables. A 256-entry Q8.8 fixed-point lookup table for sigmoid occupies only 0.5 KiB of BRAM and provides single-cycle activation computation, enabling the platform to support LSTM and GRU workloads without software function evaluation.

5.3.5 DMA-Based Data Streaming

Current bottleneck. The ARM transfers input data and weights to each core's BRAM using programmed I/O over AXI. For the Conv2 layer, the ARM writes approximately $15 \text{ KiB} \times 4 \text{ cores} = 60 \text{ KiB}$ before each layer can begin. This programmed I/O transfer is serialized and occupies the ARM processor, which cannot overlap with other computation.

Proposed solution. Use the Zynq PS DMA controller (AXI-DMA IP block) to transfer data blocks from DDR3 SDRAM to each core's BRAM:

1. ARM prepares all four core data buffers in DDR3 during the preceding layer's computation.
2. Issues four DMA transfer descriptors simultaneously, one per core.
3. The DMA engine handles AXI burst protocol, freeing the ARM for pooling computation.

Expected improvement. AXI burst transfers on the Zynq-7020 achieve approximately 1–2 GB/s (64-bit bus at 150 MHz), compared to approximately 50–100 MB/s for programmed I/O byte writes. The data-loading phase for Conv2 would reduce from approximately 1 ms to approximately 30 μs , effectively eliminating the data transfer overhead from the critical path.

5.3.6 Binary Neural Networks for Multiply-Free Inference

Concept. Binary Neural Networks (BNNs) [28] constrain all weights and activations to $\{-1, +1\}$, eliminating all multiplication operations. The MAC operation degenerates to:

$$\text{XNOR}(a_b, w_b) \longrightarrow \text{result bit}; \quad \text{popcount}(\text{result register}) \longrightarrow \text{accumulation} \quad (5.1)$$

Why this is architecturally ideal for RV32I. The RV32I ISA already provides `xor` (for XNOR) and bitwise-and operations natively. A 32-bit register holds 32 binary weights, and a single `xor` followed by a software popcount computes 32 MACs simultaneously. This approach requires no M extension, no DSP slices, and no FPU — only a new compiled worker binary with the binary-constrained kernels [29].

Accuracy trade-off. BNNs typically lose 2–5% accuracy compared to int8 models on image classification tasks. Given that the float32 FruitCNNSmall achieves >98% accuracy, a BNN variant would be expected to achieve >93% on the same task — adequate for a fruit classification application. Retraining with the straight-through estimator [28] would be required, but no hardware modifications are needed.

Summary of proposed extensions. Table 5.1 provides a comparative summary of all proposed architectural extensions.

TABLE 5.1: Summary of proposed architectural extensions for AI acceleration on the RV32I multicore platform.

Extension	HW Cost	Expected Gain	Notes
Fused MAC instruction	1 DSP/core	~10× conv speedup	Aligns with RISC-V P draft; minimal pipeline change
SIMD int8 dot4	4 DSP/core	Additional 4× throughput	16 DSPs total (7.3% of Zynq-7020)
4×4 Systolic array	16 DSP shared	~63× for Conv2	Separate IP; ARM dispatches via AXI
Hardware LU/qReLU	Re- 0 DSP	5× activation speedup	MUX in writeback stage
AXI-DMA streaming	0 DSP	~30× data load speedup	Zynq PS DMA; overlaps with compute
Binary Neural Net-work	Net- 0 DSP	32 MACs per XOR+popcount	Retraining required; no hw change

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

This report presented a controlled and reproducible evaluation of two lightweight RISC-V RV32I multicore processor designs on the PYNQ-Z2 (Zynq-7020) FPGA platform, extended with a practical CNN inference application. The contributions are summarised as follows:

- **Multicore scaling of a 4-stage RV32I pipeline.** The lightweight four-stage in-order RV32I core was extended from a single core to a multicore environment of 2, 4, and 8 cores on the PYNQ-Z2, using a unified mailbox-based synchronisation and shared BRAM framework.
- **Fair architectural comparison with a 6-stage multicycle multicore.** The proposed four-stage multicore design was compared against an existing six-stage multicycle RV32I multicore under identical runtime conditions, binaries, memory partitioning, and performance measurement methodology. The 4-stage design achieves up to $5.98\times$ speedup at 8 cores, consuming only 28,591 LUTs and 1.63 W versus 43,731 LUTs and 1.69 W for the 6-stage design.
- **Unified benchmarking framework.** A bare-metal benchmarking framework was developed using mailbox synchronisation, per-core BRAM partitioning, and hardware performance counters, enabling reproducible evaluation of MiBench workloads: Basicmath, Bitcount, Qsort, Stringsearch, FFT, IFFT, and Matmul.
- **CNN inference application.** An int8-quantized lightweight CNN (FruitC-NNSmall, $\sim 34,357$ parameters) was deployed on the 4-core system for fruit image classification across five categories. The system correctly classifies all five fruit classes, validating the multicore architecture for real-world embedded AI workloads using balanced activation calibration and a host-worker pipeline.

- **Architectural roadmap for AI acceleration.** Six concrete hardware extensions are proposed — fused MAC instruction, SIMD int8 packed operations, systolic array co-processor, hardware activation functions, DMA-based data streaming, and binary neural network support — providing a clear pathway from the current general-purpose multicore to a lightweight dedicated AI accelerator.

6.2 Limitations

The following limitations of the current work are acknowledged:

- The 4-stage core does not implement the RISC-V M extension; convolution relies on a slow software multiply routine (~ 30 cycles per multiplication).
- The 64 KiB BRAM per core limits the deployable model size to approximately 34K parameters; larger models would require external DDR3 access.
- The CNN deployment processes one pre-compiled static test image per run; no real-time camera input is integrated.
- Per-layer execution time on the RV32I workers is not instrumented; the MiBench evaluation hardware counters were not applied to the CNN layers.
- The FruitCNNSmall model was trained exclusively on the clean white-background Fruits-360 dataset; classification accuracy may degrade on cluttered real-world images.
- Soft-float arithmetic in the dequantization step accounts for a significant fraction of worker execution time, limiting the practical speedup of the CNN application.
- Scalability remains constrained by AXI interconnect contention at 8 cores, as demonstrated by sub-linear efficiency in the MiBench results.

6.3 Future Work

Future work is organised from near-term to long-term:

Near-term:

- Add the RISC-V M extension to the 4-stage pipeline to enable hardware multiply and eliminate the soft-multiply bottleneck.
- Instrument per-layer CNN execution time using the existing CSR cycle counter infrastructure.

- Integrate the PYNQ-Z2 camera interface to enable real-time image input and live inference.
- Implement AXI-DMA based data transfer from DDR3 to worker BRAMs to overlap data loading with ARM-side computation.

Medium-term:

- Add a fused MAC custom instruction using DSP48E1 slices, targeting a 10× convolution speedup.
- Implement SIMD int8 packed dot-product operations for an additional 4× throughput gain.
- Retrain FruitCNNSmall with data augmentation (random crop, flip, colour jitter) to improve robustness to real-world images.
- Extend the model to a larger subset of the Fruits-360 dataset using model compression or knowledge distillation.

Long-term:

- Design and integrate a small systolic array co-processor IP (e.g., 4×4 PE array using 16 DSP48E1 slices) for dedicated convolution acceleration.
- Explore binary neural network variants using XNOR-popcount operations natively supported by the RV32I ISA, requiring no hardware modification.
- Port the multicore system to a larger FPGA platform (e.g., Zynq UltraScale+) to increase core count and BRAM budget for deeper models.
- Compare the extended system against dedicated NPU IP cores such as the Vitis AI DPU on the same FPGA platform.

Bibliography

- [1] YosysHQ, “PicoRV32.” GitHub, <https://github.com/YosysHQ/picorv32>.
- [2] O. Kindgren, “SERV.” GitHub, <https://github.com/olofk/serv>.
- [3] S. Nolting, “NEORV32.” <https://stnolting.github.io/neorv32/>.
- [4] lowRISC, “Ibex.” GitHub, <https://github.com/lowRISC/ibex>.
- [5] ucb-bar, “riscv-sodor.” GitHub, <https://github.com/ucb-bar/riscv-sodor>.
- [6] A. Asanovic *et al.*, “The rocket chip generator,” 2016. UCB/EECS-2016-17, <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html>.
- [7] R. Holler, D. Haselberger, D. Ballek, P. Rossler, M. Krapfenbauer, and M. Linauer, “Open-source RISC-V processor IP cores for FPGAs—overview and evaluation,” in *2019 8th Mediterranean Conference on Embedded Computing (MECO)*, pp. 1–6, 2019.
- [8] M. Makni, M. Baklouti, S. Niar, M. W. Jmal, and M. Abid, “A comparison and performance evaluation of FPGA soft-cores for embedded multi-core systems,” in *2016 11th International Design & Test Symposium (IDT)*, pp. 154–159, 2016.
- [9] B. K. do Nguyen *et al.*, “Hardware software co-design for multi-threaded computation on RISC-V-based multicore system,” *IEEE Access*, vol. 12, pp. 177312–177326, 2024.
- [10] “Performance evaluation of RISC-V microcontroller system on FPGA: A study of the NEORV32 core,” *International Journal of Integrated Engineering (IJIE)*, vol. 16, no. 1, 2024.
- [11] M. Ali *et al.*, “Architectural optimization of CVA6 architecture,” 2024.
- [12] E. Lazzeri *et al.*, “An experimental comparison of RISC-V processors: Performance, power, area, and security,” 2024.
- [13] A. Kageyama *et al.*, “RVCoreP: An optimized RV32I soft processor,” 2020. arXiv:2002.03568.
- [14] B. Levy, “FemtoRV.” GitHub, <https://github.com/BrunoLevy/learn-fpga>.
- [15] PULP platform, “PULPino.” GitHub, <https://github.com/pulp-platform/pulpino>.
- [16] “Sodor core — chipyard documentation.” <https://chipyard.readthedocs.io/en/stable/Generators/Sodor.html>.

- [17] SpinalHDL, “VexRiscv.” GitHub, <https://github.com/SpinalHDL/VexRiscv>.
- [18] *PYNQ-Z2 Reference Manual v1.0*, 2018. https://www.mouser.com/datasheet/2/744/pynqz2_user_manual_v1_0-1525725.pdf.
- [19] AMD, *Zynq-7000 SoC Technical Reference Manual (UG585)*. <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>.
- [20] M. R. Guthaus *et al.*, “MiBench: A free, commercially representative embedded benchmark suite,” in *IEEE International Workshop on Workload Characterization (WWC)*, 2001.
- [21] AMD Xilinx, *Vivado Design Suite User Guide: Design Flows Overview (UG892)*, 2022.
- [22] AMD, *Getting Started with Vitis 2022.1 (UG1400)*, 2022.
- [23] H. Mureşan and M. Oltean, “Fruit recognition from images using deep learning,” *Acta Universitatis Sapientiae, Informatica*, vol. 10, no. 1, pp. 26–42, 2018.
- [24] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” 2019. NeurIPS 2019, <https://pytorch.org>.
- [25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *CVPR*, pp. 2704–2713, 2018.
- [26] “RISC-V “P” packed SIMD extension draft,” 2021. <https://github.com/riscv/riscv-p-spec>.
- [27] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.
- [28] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training deep neural networks with binary weights during propagations,” in *NeurIPS*, 2015.
- [29] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-Net: ImageNet classification using binary convolutional neural networks,” in *ECCV*, pp. 525–542, 2016.